

NEXOSuite

Sitio Web (Landing) — README del Desarrollador

Julio 2026 · Para desarrolladores frontend que reciben el proyecto
NexoHub SpA · Santiago, Chile

Para quién es esto: un desarrollador frontend al que se le entrega este sitio para agregar animaciones y terminaciones visuales. No necesitas conocimiento previo del proyecto para trabajar aquí — este documento te da todo el contexto.

Lo más importante: este sitio **no tiene backend ni build step**. Son 4 páginas HTML estáticas + un archivo CSS compartido + JS vanilla mínimo embebido en cada HTML. Lo que ves en el archivo es exactamente lo que corre en el navegador.

Si tu tarea también incluye la app del PMS/IoT (el otro repo, `nexo-room-app`), ese proyecto tiene su propio manual mucho más largo: `nexo-room-app/docs/MANUAL-DESARROLLADOR.pdf` — con sección de backend incluida. Este documento es solo para el sitio de marketing.

Estructura del proyecto

```
nexo-landing/  
├─ index.html           Home — landing principal  
├─ pms.html             Página del módulo PMS  
├─ iot.html             Página del módulo IoT  
├─ diagnostico.html     Herramienta "¿qué plan me conviene?"  
├─ politica-privacidad.html  
├─ styles/  
│   └─ nexosuite.css     Todo el CSS del sitio — un solo archivo  
└─ assets/  
    ├─ favicon-nexosuite.png  
    ├─ logo-nexosuite.svg  
    └─ gallery/           Imágenes de la galería de iot.html
```

Cada `.html` es autocontenido: trae su propio `<script>` al final del archivo con la lógica de esa página (menú hamburguesa, FAQ acordeón, animaciones de scroll). No hay un `main.js` compartido — si ves un patrón repetido en las 4 páginas (por ejemplo el observer de scroll), es intencional, cada página lo trae por separado para no depender de un archivo externo.

Cómo trabajar en local

No necesita instalación. Dos formas:

1. **Directo:** doble clic en cualquier `.html` y ábrelo en el navegador.
2. **Con servidor local** (recomendado si vas a tocar rutas entre páginas):

```
npx serve .
```

o la extensión "Live Server" de VSCode (recarga automática al guardar).

Sistema de diseño

Todo el CSS vive en `styles/nexosuite.css`, con variables definidas en `:root`:

```
:root {  
  --c-bg:      #F8F7F4;  /* fondo general */
```

```

--c-surface:    #F0EDE6; /* fondo de secciones alternas */
--c-teal-dark:  #085041; /* verde oscuro – marca principal */
--c-teal:       #1D9E75; /* verde medio */
--c-teal-light: #E1F5EE;
--c-text:       #111111;
--c-text-2:     #4A4845; /* texto secundario */
--c-white:      #FFFFFF;

--c-pms:        #185FA5; /* azul – usado solo en el módulo PMS */
--c-iot:        #085041; /* verde – usado solo en el módulo IoT */

--font-display: 'Fraunces', Georgia, serif; /* títulos */
--font-body:    'DM Sans', system-ui, sans-serif; /* cuerpo de texto */
}

```

Usa siempre estas variables (`var(--c-teal)` , etc.) en vez de escribir el color hexadecimal directo — así cualquier ajuste de paleta futuro se hace en un solo lugar.

Hay también variables de espaciado responsive (`--section-py`) que cambian en el media query de mobile — no dupliques media queries si puedes ajustar la variable en su lugar.

Animaciones ya existentes — el patrón .reveal

El sitio ya tiene un sistema de animación de entrada al hacer scroll. **Reutilízalo en vez de crear uno nuevo:**

```

/* En nexosuite.css */
.reveal { opacity: 0; transform: translateY(16px); transition: opacity 0.5s ease, transform 0.5s ease; }
.reveal.visible { opacity: 1; transform: none; }
.reveal-delay-1 { transition-delay: .1s; }
.reveal-delay-2 { transition-delay: .2s; }
.reveal-delay-3 { transition-delay: .3s; }

```

```

<!-- En cualquier HTML -->
<div class="card reveal reveal-delay-1">...</div>

```

Al final de cada `.html` hay un `IntersectionObserver` que agrega la clase `.visible` cuando el elemento entra en pantalla — así dispara la transición. Para animar un elemento nuevo, **solo agrégale la clase `reveal`** (y opcionalmente `reveal-delay-1/2/3` para escalar varios elementos que aparecen juntos) — no necesitas tocar el JavaScript del observer, ya está buscando cualquier `.reveal` de la página.

Para agregar una animación distinta (no solo fade-in-up)

Puedes crear variantes nuevas siguiendo el mismo patrón, por ejemplo:

```

.reveal-scale { opacity: 0; transform: scale(.95); transition: opacity 0.5s ease, transform 0.5s ease; }
.reveal-scale.visible { opacity: 1; transform: none; }

```

Y en el HTML usar `class="card reveal-scale"` en vez de `class="card reveal"` . El observer existente en cada página busca la clase `reveal` — si usas un nombre de clase distinto para la variante, revisa el `<script>` al final del HTML y agrega tu selector nuevo al `querySelectorAll` del observer (es una sola línea, está al final de cada archivo).

Otros patrones de JS que ya existen (no los dupliques)

Cada página trae, en su `<script>` final:

- **Menú hamburguesa** (`#ham` / `#nav-links`) — toggle de clase `.open` .
- **FAQ acordeón** (`.faq-btn` / `.faq-answer`) — mismo patrón en las 4 páginas.
- **Highlight del link activo en el nav** según la URL actual.
- **Scroll suave** para links internos (`a[href^="#"]`).

Si necesitas agregar interactividad nueva, revisa primero si ya existe algo parecido en ese mismo `<script>` — es corto (menos de 50 líneas) y fácil de leer completo.

Checklist antes de cada push

Este repo **también tiene auto-deploy**: cualquier push a `main` se publica solo en GitHub Pages (nexosuite.cl / el dominio de GitHub Pages configurado), sin ambiente intermedio.

- ☐ Probé el cambio abriendo el `.html` en el navegador (no solo lo leí en el código)
- ☐ Usé variables CSS (`var(--c-...)`) en vez de colores hardcodeados
- ☐ Si es una animación de scroll, usé la clase `.reveal` existente en vez de crear un observer nuevo
- ☐ Probé en mobile (o con las devtools en modo responsive) — el sitio tiene un layout distinto bajo cierto ancho de pantalla (ver el media query en `nexosuite.css`)
- ☐ No agregué librerías externas (jQuery, GSAP, etc.) sin avisar — el sitio es deliberadamente liviano, sin dependencias

NexoSuite · Sitio Web · Julio 2026